The DEK is then itself encrypted using the KMS. The encrypted DEK can then be safely stored or transmitted alongside the encrypted data. To decrypt the recipient first decrypts the DEK using the KMS and then uses the DEK to decrypt the rest of the data.

> **DEFINITION** In ***envelope encryption*** an application encrypts data with a local ***data encryption key (DEK)***. The DEK is then encrypted (or *wrapped*) with a ***key encryption key (KEK)*** stored in a KMS or other secure service. The KEK itself might be encrypted with another KEK creating a ***key hierarchy***.

For both secrets management and KMS, the client usually interacts with the service using a REST API. Currently, there is no common standard API supported by all providers. Some cloud providers allow access to a KMS using the standard PKCS#11 API used by hardware security modules. You can access a PKCS#11 API in Java through the Java Cryptography Architecture, as if it was a local keystore, as shown in listing 11.13. Java exposes a PKCS#11 device, including a remote one such as a KMS, as a `KeyStore` object with the type "PKCS11".[7] You can load the keystore by calling the `load()` method, providing a null `InputStream` argument (because there is no local keystore file to open) and passing the KMS password or other credential as the second argument. After the PKCS#11 keystore has been loaded, you can then load keys and use them to initialize `Signature` and `Cipher` objects just like any other local key. The difference is that the `Key` object returned by the PKCS#11 keystore has no key material inside it. Instead, Java will